This approach provides flexibility, especially when the amount of memory needed isn't known beforehand. The C standard library offers functions for dynamic memory management: `malloc()`, `calloc()`, and `free()`.

`malloc()` (Memory Allocation)

Allocates a specified number of bytes in memory and returns a pointer to the first byte.

```
int *ptr = (int*) malloc(sizeof(int) * 5);  // Allocates memory for 5 integers
```

- **Important:** Always cast the returned `void*` to the appropriate type.
- **Failure Check:** `malloc()` returns `NULL` if the allocation fails.

`calloc()` (Contiguous Allocation)

Similar to `malloc()` but initializes the allocated memory to zero.

```
int *ptr = (int*) calloc(5, sizeof(int));  // Allocates and zeros out memory for 5 integers
```

`free()`

Deallocates previously allocated memory, preventing memory leaks.

```
free(ptr);

ptr = NULL;  // Best practice to avoid dangling pointers
```